RELATÓRIO EXPERIMENTO 3

Nome: Renato Gabriel Lima da Silva
Matrícula: 251027871

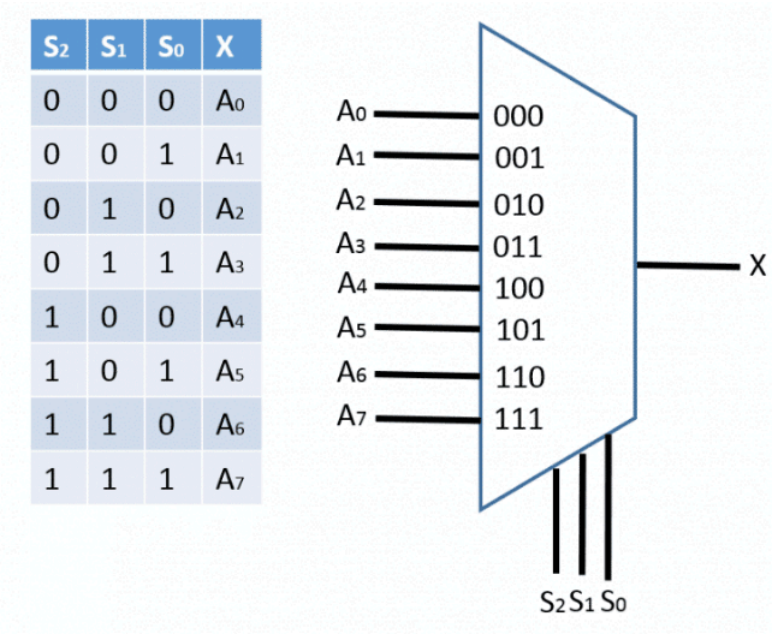
INTRODUÇÃO

Neste experimento, iremos abordar algumas entidades utilizando atribuições condicionais e atribuições seletivas da linguagem de descrição de hardware VHDL e simular no software ModelSim, sendo elas um Multiplexador 8 para 1 e um Decodificador 4 para 16, vamos abordar seus conceitos e suas funcionalidades.

TEORIA

1 - Multiplexador

No primeiro exercício vamos implementar um Multiplexador 8x1, este circuito e um seletor de entradas, portanto o valor da saída (X) vai depender da escolha feita pelas entradas de seleção (S_0,S_1 e S_2) das entradas de dados (A_0, A_1, A_2, A_3, … e A_7) Ex: caso S_0 S_1 e S_2 sejam 0 a saída será o valor apresentado em A_0.
Segue o circuito do Multiplexador e sua tabela verdade:



S_2	S_1	S_0	X
0	0	0	A_0
0	0	1	A_1
0	1	0	A_2
0	1	1	A_3
1	0	0	A_4
1	0	1	A_5
1	1	0	A_6
1	1	1	A_7

Figura 1 : exemplo de um Multiplexador e tabela verdade.

2 - Decodificador

No segundo exercício vamos implementar um Decodificador 4x16, Um decodificador é um dispositivo ou circuito lógico que converte um sinal codificado (compactado ou em formato binário) de volta para um formato reconhecível e utilizável pelo utilizador ou sistema. Nesse caso, possuindo 4 bits de entrada (A) e 1 vetor de 16 bits de saída (Y). Já as atribuições seletivas são uma seleção de entradas para que dependendo delas você obtenha a saída correspondente. A Tabela 1 apresenta a tabela-verdade desejada para a implementação feita na atividade possuindo a coluna com todos os possíveis valores de entrada (A) e mais uma coluna com os valores de saída apresentados da saída (Y):

Tabela 1:

A	*Y*
0000	0000000000000001
0001	0000000000000010
0010	0000000000000100
0011	0000000000001000
0100	0000000000010000
0101	0000000000100000
0110	0000000001000000
0111	0000000010000000
1000	0000000100000000
1001	0000001000000000
1010	0000010000000000
1011	0000100000000000
1100	0001000000000000
1101	0010000000000000
1110	0100000000000000
1111	1000000000000000

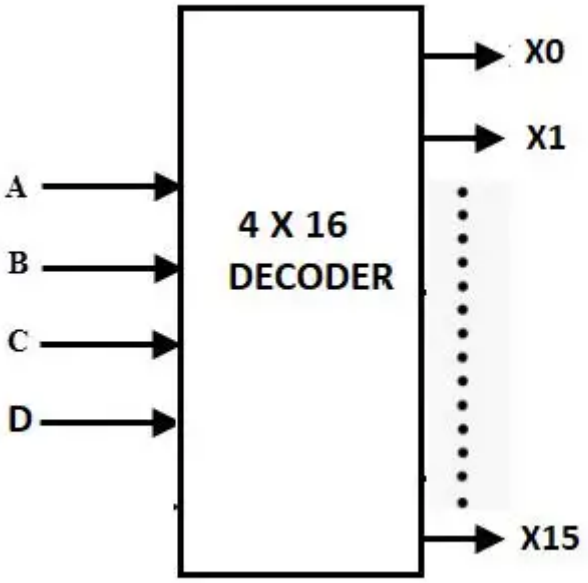


Figura 2 : exemplo de um decodificador.

CÓDIGOS

1 - Multiplexador

Agora vamos analisar a implementação do código do multiplexador:

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3
4
5   entity meucircuitol is
6       port (S: in std_logic_vector(2 dow
7       D: in std_logic_vector(7 downto 0)
8       Y: out std_logic);
9   end meucircuitol;
10
11
12  architecture meucircuitol_arch of meuc
13  begin
14      Y <= D(0) when (S = "000") else
15           D(1) when (S = "001") else
16           D(2) when (S = "010") else
17           D(3) when (S = "011") else
18           D(4) when (S = "100") else
19           D(5) when (S = "101") else
20           D(6) when (S = "110") else
21           D(7);
22  end meucircuitol_arch;
```

Figura 3 : Código do Multiplexador.

A primeira e segunda linha permitem o uso dos tipos STD_LOGIC e STD_LOGIC_VECTOR no código, onde para essa atividade utilizamos ambas, pois nesse caso as entradas são vetores e a saída não.

As linhas 4 à 8 definem uma entidade com nome “meucircuito1”, que é onde é definido as variáveis de entrada (in) e saída (out), como as entradas são vetores além de utilizar a STD_LOGIC_VECTOR é necessário também especificar o tamanho e se seu índice é crescente (TO) ou decrescente (DONWTO), neste caso a variável D e S são ambas decrescente,ou seja e S possui 3 bits mesmo aparecendo apenas o número 2, pois o índice 0 já é contado quando utilizamos um vetor, logo é necessário adicionar um a menos quando se declara o tamanho. O mesmo vale para a variável D onde ele possui 8 bits mas é representado com o número 7. Já a saída Y como é apenas um bit é implementada com a STD_LOGIC normalmente. As entradas e saídas da entidade são definidas conforme a discussão anterior: Sendo Um vetor D com 8 bits e um vetor S com 3 bits na entrada, e uma Y saída de 1 bit

Por fim, a arquitetura é definida entre as linhas 10 a 20 usando as atribuições de condicionais onde ela é reconhecida nas linhas 12 a 19 por conta dos nomes “when” e “else” e que funciona da seguinte maneira. A saída Y vai ser a entrada D_n de acordo com o valor de S como na linha 12 onde ela fala que: “Y vai ser igual a D_0 quando S for igual a 000”. Caso isso não ocorra, ele vai para a próxima afirmação, por meio do else, sendo no caso a linha 13 e assim sucessivamente até que ele encontre o S correspondente a entrada caso não encontre, Y vai ser igual a D_7 que no caso seria a entrada de S igual a 111.

Esta implementação nos permite observar e conferir a tabela-verdade apresentada anteriormente (Figura 1). Sendo praticamente uma implementação mais literal da tabela-verdade apresentada.

2 - Decodificador 4x16

Agora vamos analisar a implementação do código do Decodificador:

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3
4
5   entity meucircuito2 is
6       port (A: in std_logic_vector(3 downto 0);
7       Y: out std_logic_vector(15 downto 0));
8   end meucircuito2;
9
10
11  architecture meucircuito2_arch of meucircuito2 is
12  begin
13      with A select
14      Y <= "0000000000000001" when "0000",
15           "0000000000000010" when "0001",
16           "0000000000000100" when "0010",
17           "0000000000001000" when "0011",
18           "0000000000010000" when "0100",
19           "0000000000100000" when "0101",
20           "0000000001000000" when "0110",
21           "0000000010000000" when "0111",
22           "0000000100000000" when "1000",
23           "0000001000000000" when "1001",
24           "0000010000000000" when "1010",
25           "0000100000000000" when "1011",
26           "0001000000000000" when "1100",
27           "0010000000000000" when "1101",
28           "0100000000000000" when "1110",
29           "1000000000000000" when others;
30  end meucircuito2_arch;
```

A primeira e segunda linha são iguais às apresentadas no código anterior. As linhas 4 à 7 definem uma entidade com nome “meucircuito2”, que é bem semelhante a apresentada na listagem 2 onde é definido as variáveis de entrada (in) e saída (out), com a entrada A sendo um vetor com 4 bits decrescentes e a saída Y sendo um vetor com 16 bits.

Por fim, a arquitetura é definida entre as linhas 9 a 28 usando a atribuição seletivas. onde Y vai ser um valor que depende do valor selecionada de A, como na linha 12 onde Y é igual a 0000000000000001 quando o valor selecionado de A for igual a 0000. Sendo feito dessa forma até a linha 27 onde Y vai ser 1000000000000000 quando A for igual a qualquer outra entrada que não seja as citadas anteriormente no caso como só sobrou uma será quando A for igual a 1111. Esta implementação nos permite observar e conferir a tabela-verdade apresentada anteriormente (Tabela 1).

COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:

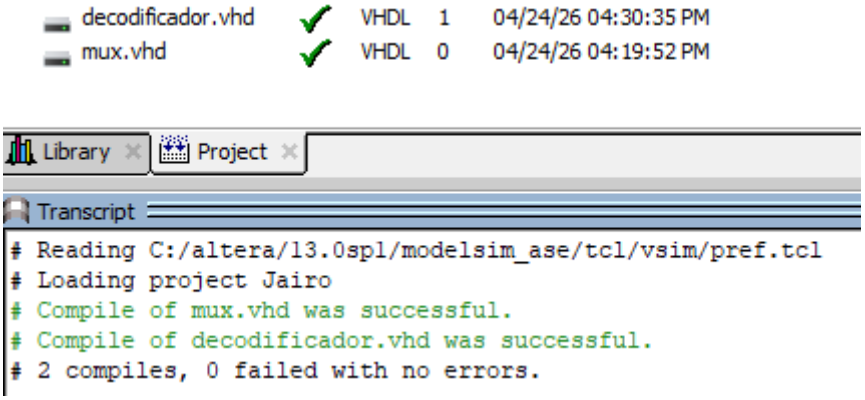


Figura 2: Compilação dos códigos

Como visto na figura 2, nenhum código tem algum erro de sintaxe.

SIMULAÇÃO E ANÁLISE

1- Multiplexador

Para verificar a correção do código implementado, ele foi simulado no software Modelsim. Para a simulação, foi feito um testbench com 2 valores diferentes para as variáveis D e um for para ir mudando o valor das variáveis S a cada 10 ns passando por todos os casos de S, o primeiro valor pra D sendo 10101010 e após passar por todos os casos do bit de seleção ele é mudado para 11110000. Segue o código do testbench:

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.numeric_std.all;
4
5   entity tb_multiplexador is
6   end tb_multiplexador;
7
8   architecture behavior of tb_multiplexador is
9       signal S_tb : std_logic_vector(2 downto 0) := "000";
10      signal D_tb : std_logic_vector(7 downto 0) := "10101010";
11      signal Y_tb : std_logic;
12
13  begin
14
15      uut: entity work.meucircuito1
16          port map (
17              S => S_tb,
18              D => D_tb,
19              Y => Y_tb
20          );
21
22      stim_proc: process
23      begin
24          for i in 0 to 7 loop
25              S_tb <= std_logic_vector(to_unsigned(i, 3));
26              wait for 10 ns;
27          end loop;
28
29          D_tb <= "11110000";
30          for i in 0 to 7 loop
31              S_tb <= std_logic_vector(to_unsigned(i, 3));
32              wait for 10 ns;
33          end loop;
34          wait;
35      end process;
36  end behavior;
```

Com isso conseguimos simular por meio do ModelSim e observar os resultados por meio da forma de onda apresentada:

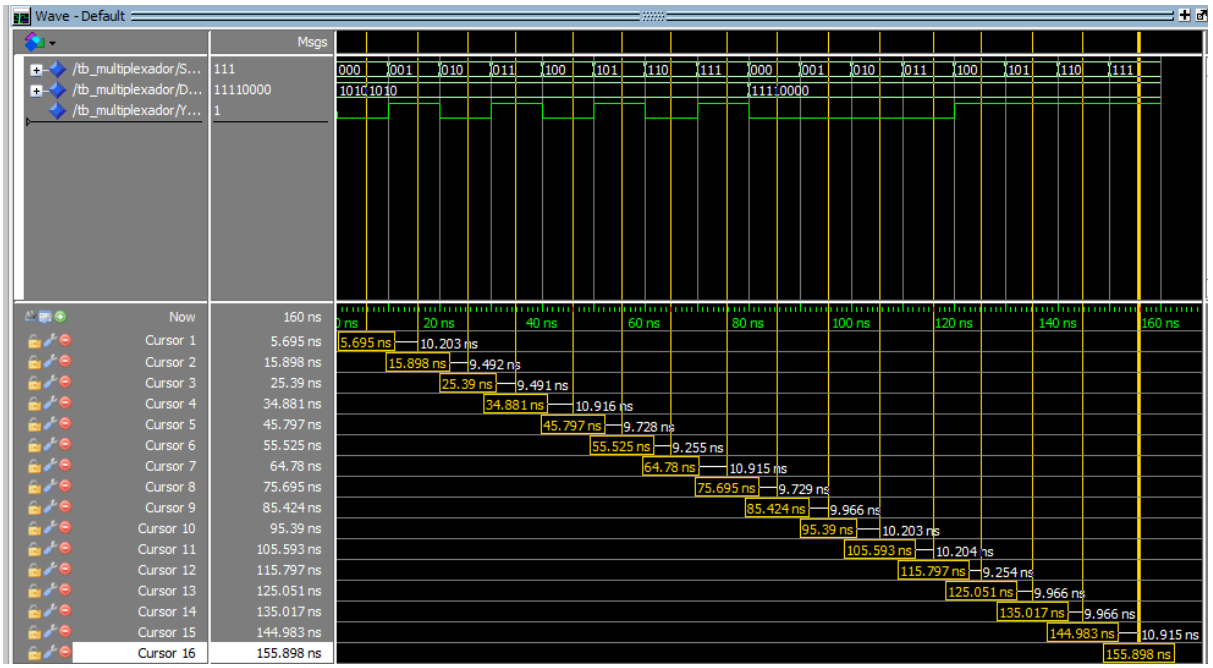


Figura 3: Simulador do mux.

Na figura 3 temos as formas de onda e 16 cursor onde nela podemos observar o valor de saída de cada seleção:

- Cursor 1: D=10101010, S=000 e Y=0
- Cursor 2: D=10101010, S=001 e Y=1
- Cursor 3: D=10101010, S=010 e Y=0
- Cursor 4: D=10101010, S=011 e Y=1
- Cursor 5: D=10101010, S=100 e Y=0
- Cursor 6: D=10101010, S=101 e Y=1
- Cursor 7: D=10101010, S=110 e Y=0
- Cursor 8: D=10101010, S=111 e Y=1
- Cursor 9: D=11110000, S=000 e Y=0
- Cursor 10: D=11110000, S=001 e Y=0
- Cursor 11: D=11110000, S=010 e Y=0
- Cursor 12: D=11110000, S=011 e Y=0
- Cursor 13: D=11110000, S=100 e Y=1
- Cursor 14: D=11110000, S=101 e Y=1
- Cursor 15: D=11110000, S=110 e Y=1
- Cursor 16: D=11110000, S=111 e Y=1

Com isso, podemos ver as seleções corretas do bit de seleção S para os valores D, provando que o MUX foi implementado corretamente e está funcionando como o esperado.

2 - Decodificador

Para verificar a correção do código implementado, ele foi simulado no software Modelsim. Para a simulação, foi feito um testbench com todas as combinações possíveis para a entrada A variando os valores a cada 10 ns, com isso podemos ver se a tabela apresentada é a mesma que a tabela apresentada anteriormente (Tabela 1). Segue o código do testbench:

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.numeric_std.all;
4
5   entity tb_decoder is
6   end tb_decoder;
7
8   architecture behavior of tb_decoder is
9
10      signal A_tb : std_logic_vector(3 downto 0) := "0000";
11      signal Y_tb : std_logic_vector(15 downto 0);
12
13  begin
14
15      uut: entity work.meucircuito2
16          port map (
17              A => A_tb,
18              Y => Y_tb
19          );
20      stim_proc: process
21      begin
22          for i in 0 to 15 loop
23              A_tb <= std_logic_vector(to_unsigned(i, 4));
24              wait for 10 ns;
25          end loop;
26          wait;
27      end process;
28  end behavior;
```

Com isso conseguimos simular por meio do ModelSim e observar os resultados por meio da forma de onda apresentada:

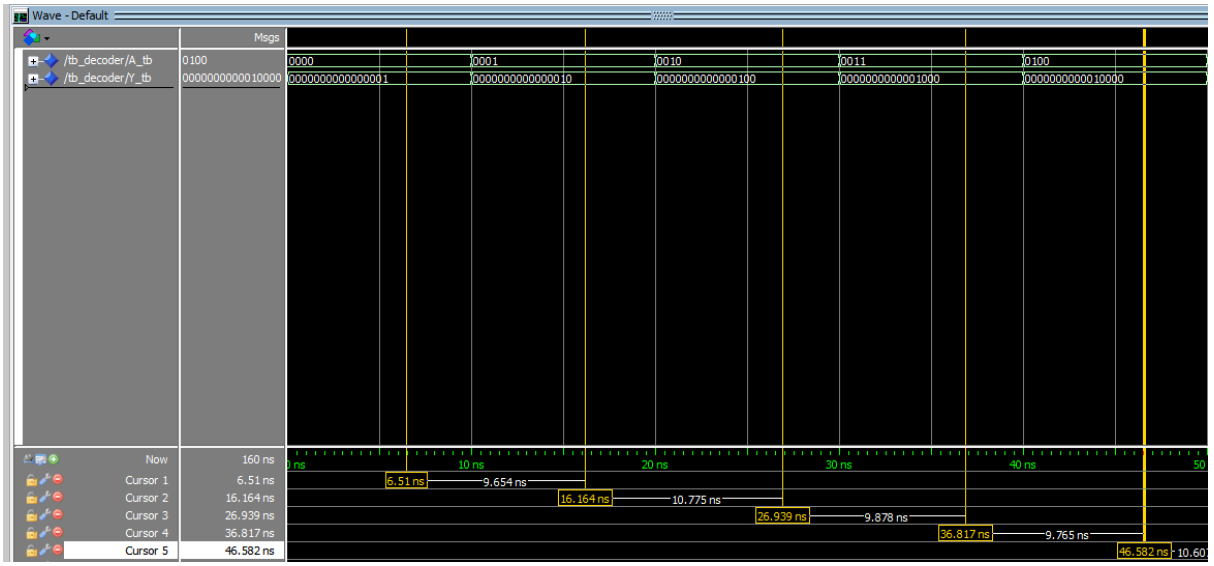


Figura 4.1: Simulação decodificador em forma de onda binária (cursor 1 ao 5)

Figura 4.2: Simulação decodificador em forma de onda binária (cursor 6 ao 10)

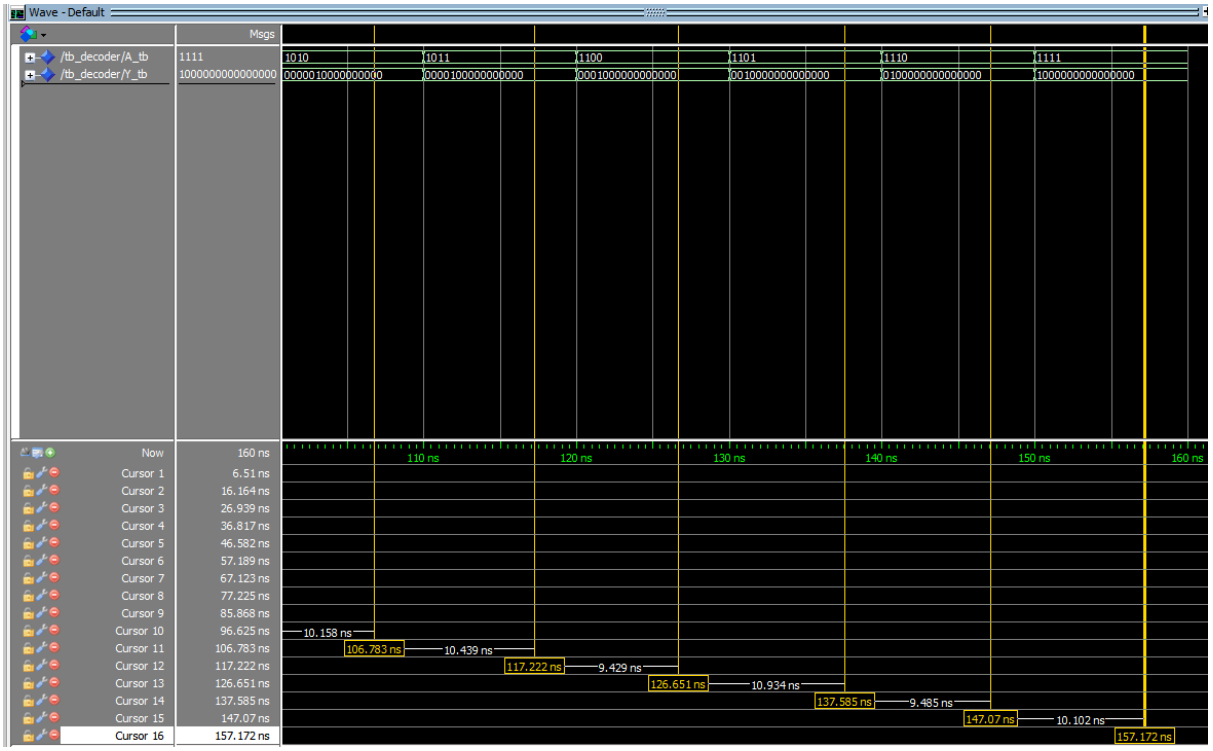


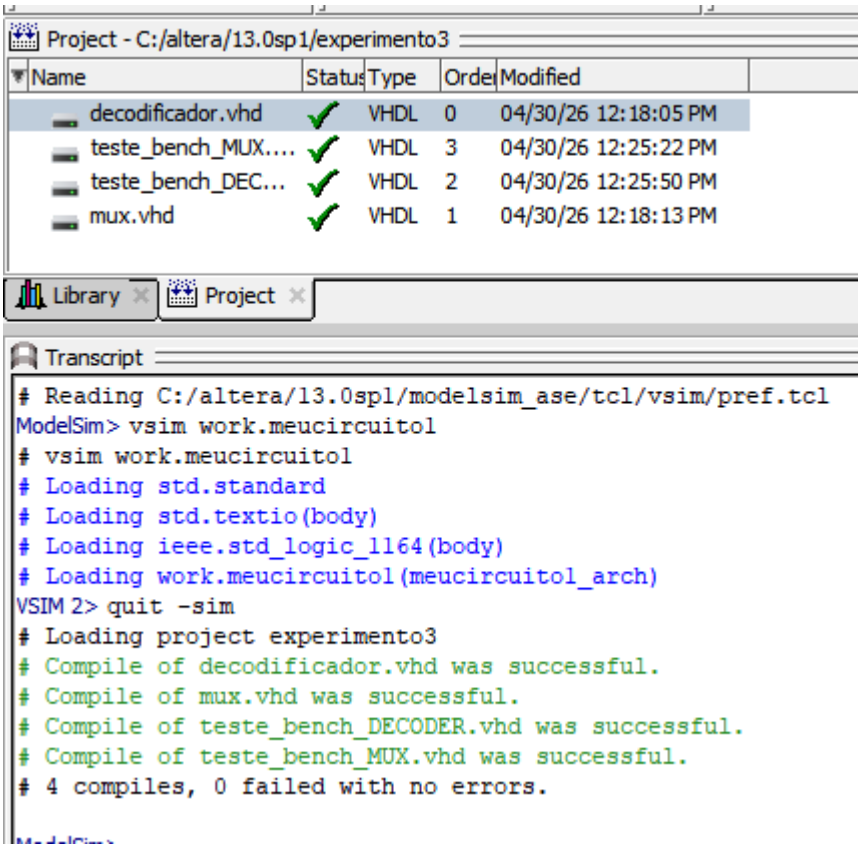
Figura 4.3: Simulação decodificador em forma de onda binária (cursor 11 ao 16)

Cada cursor está em um valor de A e sua respectiva saída ficando da seguinte forma:

- Cursor 1: A = 0000 e Y = 0000000000000001
- Cursor 2: A = 0001 e Y = 0000000000000010
- Cursor 3: A = 0010 e Y = 0000000000000100
- Cursor 4: A = 0011 e Y = 0000000000001000
- Cursor 5: A = 0100 e Y = 0000000000010000
- Cursor 6: A = 0101 e Y = 0000000000100000
- Cursor 7: A = 0110 e Y = 0000000001000000
- Cursor 8: A = 0111 e Y = 0000000010000000
- Cursor 9: A = 1000 e Y = 0000000100000000
- Cursor 10: A = 1001 e Y = 0000001000000000
- Cursor 11: A = 1010 e Y = 0000010000000000
- Cursor 12: A = 1011 e Y = 0000100000000000
- Cursor 13: A = 1100 e Y = 0001000000000000
- Cursor 14: A = 1101 e Y = 0010000000000000
- Cursor 15: A = 1110 e Y = 0100000000000000
- Cursor 16: A = 1111 e Y = 1000000000000000

Com isso, podemos observar que ficou exatamente como a tabela 1, com isso podemos provar que foi implementado um decoder 4x16 da maneira correta.

Ambos os testbench foram compilados no ModelSim também:



CONCLUSÃO

Portanto, neste experimento conseguimos conhecer as atribuições condicionais e atribuições seletivas da linguagem VHDL e conseguimos implementar os circuitos corretamente, conseguindo simulá los com várias combinações sendo eles o Mux e o Decoder.